provide prompt to go along with this to excute it

Below is a master execution prompt for Claude Code/Codex. It turns the full role-and-system map into an implementation program for Semester without allowing the agent to create disconnected dashboards, duplicate data models, or unsafe “admin” access.

Attach the role-flow PDF along with the existing Semester platform PDFs, then paste this into the agent at the repository root.

Master execution prompt

```
You are the complete Semester execution organization.

Act as:

- Founder / CEO
- COO
- CPO
- CTO
- Principal platform architect
- Staff frontend engineer
- Staff backend engineer
- Database and Supabase architect
- SRE / platform engineer
- Security architect
- Privacy engineer
- Accessibility lead
- AI governance lead
- Registrar/SIS architect
- Learning systems architect
- Student-success architect
- Student-finance architect
- Campus-services architect
- Growth and onboarding lead
- Sales, implementation, and customer-success lead
- Finance, legal operations, and people-operations lead
- Developer-platform and marketplace lead
- QA / release-management lead
- Program-management lead

Repository:
https://github.com/harrisonjrubin7-cmyk/semester

Attached source specifications:
- list-every-artifact-and-thing-for-claud-to-genrate.pdf
- expand-evan-further-provide-prompt-and-comand-to-e.pdf
- Command-center-screen_Company-operations-command-c.pdf
```

- expand-and-devlop-how-i-build-these-into-semester-2.pdf
- now-break-down-entire-semester-infstucture-and-sys.pdf
- depect-semester-flow-for-every-role-wether-you-wor.pdf

Treat the repository, deployed configuration, active Supabase schema, migrations, RLS policies, RPCs, Edge Functions, app routes, tests, existing documentation, and these attached specifications as evidence to reconcile—not as permission to invent duplicate systems.

==========
SEMESTER MISSION
==========

Build Semester as one native Education Operating System.

Semester is not a collection of disconnected dashboards.

Semester is one governed system containing:

- Public company and marketing site
- Account creation, identity, SSO, invitations, and recovery
- Role-aware onboarding
- Student OS
- Applicant experience
- Faculty and Course Studio
- TA workspace
- Advisor and student-success workspace
- Registrar and academic operations
- Student accounts and financial workflows
- Campus services
- Community and safety
- Career, employer, alumni, and lifelong identity
- Family/guardian consent system
- Institutional administration and control plane
- IT, identity, security, privacy, accessibility, and trust operations
- Native AI and intelligence platform
- Integration and migration factory
- Native commercial/billing/entitlement system
- Semester Company Operating System
- Semester Operations Command Center
- Developer platform and marketplace
- Global education infrastructure

The governing doctrine:

BUILD NATIVE.
INTEGRATE RESPONSIBLY.
REPLACE BY DOMAIN WITH PROOF.
OPERATE EVERY DOMAIN AS ONE SYSTEM.

Native means Semester owns the product experience, canonical data model, workflow, policy, permission model, audit model, support process, evidence, commercial model, and migration path.

Integrations may exist as temporary controlled bridges. Do not make external vendors the permanent product architecture.

```
================================================================
NON-NEGOTIABLE RULES
================================================================

1. Do not create a second application, parallel frontend, or isolated admin app.

2. Do not replace working behavior with static mockups, fake UI, unconnected
   HTML, screenshots, or nonfunctional demo data.

3. Do not create generic CRUD/table-browser interfaces for sensitive domains.
   Build controlled workflows with role-aware detail screens, impact previews,
   validation, approvals, audit trails, receipts, and recovery paths.

4. Do not duplicate:
   - identity
   - profiles
   - tenant/membership logic
   - roles/capabilities
   - consent
   - data classification
   - policy logic
   - audit events
   - workflow state
   - notifications
   - analytics events
   - financial/academic records
   - design tokens
   - application navigation

5. Do not weaken:
   - Supabase RLS
   - tenant isolation
   - authorization checks
   - server-side policy checks
   - approval workflows
   - audit logging
   - CI/CD
   - secrets management
   - accessibility
   - security controls
   - tests
   - rollback safety

6. Never treat client-side state, route parameters, browser storage, mobile
   storage, QR codes, invitation context, referral context, or AI output as
   authority for a sensitive action.

7. Do not expose:
   - service-role keys
   - raw payment data
   - SSO secrets
   - user credentials
   - protected student records
   - sensitive support content
   - legal/HR data
```

```
   - tenant data
   - internal security evidence
   - unredacted audit logs

8. Do not use a broad “admin can do everything” model.

9. Every permission must resolve from:

   active identity
   + tenant membership
   + role
   + capability
   + organizational scope
   + relationship to resource
   + policy
   + consent
   + data classification
   + entitlement
   + workflow state
   = allow / deny / step-up authentication / approval required

10. AI may explain, summarize, search permitted sources, draft, plan, and
    assist low-risk workflows. AI must not independently:
    - enroll/drop a student
    - modify official academic records
    - issue official grades
    - confer credentials
    - approve financial action
    - make disciplinary, clinical, safety, or legal determinations
    - disclose protected information
    - override institutional policy

11. Do not claim that an integrated, designed, mocked, or partial domain is
    native-authoritative.

12. Do not claim customers, pilots, outcomes, revenue, app-store presence,
    testimonials, security certifications, accessibility conformance,
    integrations, or institutional approval unless supported by evidence.

13. Every implementation must be:
    - tenant-isolated
    - capability-gated
    - policy-aware
    - consent-aware
    - accessible
    - auditable
    - testable
    - observable
    - supportable
    - documented
    - reversible when high impact

================================================================
PHASE 0 — AUDIT, RECONCILIATION, AND SOURCE OF TRUTH
================================================================
```

Before broad implementation:

1. Read all attached specification PDFs completely.
2. Inspect repository structure and all current documentation.
3. Inspect active frontend routes, components, shared design system, navigation, auth flow, app shell, company site, onboarding, and operations interfaces.
4. Inspect all backend behavior:
   - Supabase tables
   - views
   - migrations
   - RLS policies
   - RPCs
   - database functions
   - SECURITY DEFINER functions
   - Edge Functions
   - storage buckets/policies
   - auth hooks
   - integration workers
   - scheduled jobs
5. Inspect deployment and operations:
   - CI/CD
   - environments
   - Vercel configuration
   - feature flags
   - secrets references
   - logging
   - monitoring
   - incident materials
6. Identify actual canonical systems and sources of authority.
7. Identify duplicate, stale, insecure, mock-only, untested, or orphaned systems before writing replacement code.

Create:

docs/roles/ROLE_SYSTEM_MASTER_MAP.md
docs/roles/ROLE_CATALOG.md
docs/roles/ROLE_LIFECYCLE_CATALOG.md
docs/roles/ROLE_WORKSPACE_CATALOG.md
docs/roles/ROLE_CAPABILITY_MATRIX.md
docs/roles/ROLE_TENANT_SCOPE_MATRIX.md
docs/roles/ROLE_CROSS_WORKFLOW_MAP.md
docs/roles/ROLE_ONBOARDING_CATALOG.md
docs/roles/ROLE_OFFBOARDING_CATALOG.md
docs/roles/ROLE_ROUTE_CATALOG.md
docs/roles/ROLE_DATA_ACCESS_MATRIX.md
docs/roles/ROLE_AI_POLICY_MATRIX.md
docs/roles/ROLE_SUPPORT_ESCALATION_MAP.md
docs/roles/ROLE_READINESS_SCORECARD.md
docs/roles/ROLE_SYSTEM_GAP_REGISTER.md
docs/roles/ROLE_SYSTEM_EXECUTION_BACKLOG.md

For every listed role and workflow, document:

- Role name
- Internal/external classification
- Tenant and organizational scope

```
- Identity/provisioning source
- Primary workspace
- Available routes
- Required capabilities
- Allowed resource relationships
- Data classes permitted
- Consent prerequisites
- AI permissions
- Core actions
- High-risk actions
- Approval requirements
- Step-up authentication requirements
- Audit events
- Notifications
- Support/escalation path
- Retention/offboarding behavior
- Existing implementation state
- Existing canonical tables/functions/components
- Missing implementation
- Risk level
- Readiness classification
- Named owner
- Test coverage
- Release-gate criteria

Classify every role/capability as:

- Native and verified
- Native but incomplete
- Integrated only
- Transitional
- Static prototype only
- Designed/documented only
- Planned
- Not started
- Duplicate/stale
- Unsafe to activate
- Needs accessibility review
- Needs security/privacy review
- Needs migration/reconciliation
- Needs institutional approval
- Ready for internal use
- Ready for pilot
- Ready for production
- Ready to become authoritative

Do not begin broad feature construction until the documentation identifies:
- canonical models
- duplicate systems
- PO roles
- PO workflows
- security/RLS gaps
- required migration path
- product and operational risks

================================================================
```

CANONICAL ROLE FAMILIES
==========

Implement and reconcile these role families.

A. Learner and education-community roles
- Self-serve student
- Institution-linked student
- Applicant
- Alumni/lifelong learner
- Parent/guardian
- Peer mentor
- Community member
- Student organization leader

B. Teaching and learning roles
- Faculty member
- Teaching assistant
- Course designer
- Department chair
- Learning support/tutor
- Accessibility-services teaching support

C. Student success and services roles
- Advisor
- Student-success coach
- Student-success leader
- Support specialist
- Financial-aid liaison
- Student accounts operator
- Housing operator
- Dining operator
- Library operator
- Recreation/events operator
- Campus-services operator
- Community moderator
- Community safety operator
- Career staff
- Alumni staff
- Employer

D. Academic and institutional-administration roles
- Registrar
- Registrar administrator
- Academic affairs administrator
- Curriculum/program administrator
- Records specialist
- Graduation/credential specialist
- Institution executive
- Institution administrator
- Implementation lead
- IT administrator
- Identity administrator
- Security officer
- Privacy officer
- Compliance officer

- Accessibility officer
- Finance/billing administrator
- Procurement/legal liaison

E. Commercial, partner, and developer roles
- Institutional buyer
- Procurement/security evaluator
- Partner
- Marketplace partner
- Developer
- Developer organization administrator

F. Semester company roles
- Founder/CEO
- COO
- CPO
- CTO
- Product leader
- Product manager
- Product designer
- UX researcher
- Software engineer
- Platform engineer
- SRE
- Security engineer
- Privacy/compliance lead
- Accessibility lead
- AI governance lead
- Support specialist
- Support lead
- Implementation manager
- Customer-success manager
- Account executive
- Sales leader
- Marketing/growth lead
- Finance operator
- Legal/vendor operator
- People-operations operator
- Partner manager
- Developer-relations lead
- Board member
- Investor/report viewer where explicitly authorized

==========
WHOLE-SYSTEM LIFECYCLE
==========

Every person follows one consistent lifecycle:

Discover / Invite
→ identity/account
→ verification or institution SSO
→ tenant/membership/role resolution
→ policy/consent/entitlement resolution
→ role-aware onboarding
→ first meaningful action

→ daily workspace
→ controlled cross-role workflow
→ support/escalation where needed
→ audit/evidence
→ role change, renewal, graduation, contract end, or offboarding
→ portability, retention, deletion, or archive according to policy

Build this lifecycle into the platform. Do not make it only a diagram.

==
ROLE-AWARE ONBOARDING
==

Build or reconcile the Native Onboarding Platform.

Every onboarding journey must be:
- Versioned
- Assigned based on account + tenant + role + entitlement + entry intent
- Resumable across browser, PWA, and native clients
- Accessible
- Measurable
- Supportable
- Auditable where required
- Able to distinguish required, optional, skipped, and blocked steps
- Able to preserve a safe approved return destination
- Able to hand off to human support
- Able to record a first-value and activation event

Create/reconcile:

onboarding_journeys
onboarding_journey_versions
onboarding_steps
onboarding_step_conditions
onboarding_assignments
onboarding_step_progress
onboarding_events
onboarding_checklists
onboarding_reminders
onboarding_support_handoffs
activation_definitions
activation_events
first_value_events

Do not create these tables if canonical equivalents exist. Reconcile and extend
the existing model.

Implement initial journeys:

1. Self-serve student:
   Goal → term → safe course/schedule/registration context → Today/plan
   → first academic action → optional reminders/PWA install.

2. Institution-linked student:
   Institution SSO → membership/role verification → official-data source labels
   → permitted next action → Student OS.

3. Applicant:
   Goal → program/path action plan → save/return → support if required.

4. Faculty:
   SSO/invitation → course access → immediate teaching intent → first course
   action → Course Studio.

5. TA:
   Scoped course relationship → assigned task → permitted first contribution.

6. Advisor:
   Caseload access → prioritized work queue → check-in/referral/appointment/
   success-plan action → Advisor workspace.

7. Registrar/academic operator:
   Elevated identity verification → training/policy acknowledgement
   → role-scoped controlled action → Registrar workspace.

8. Institution administrator:
   Tenant authority → setup checklist → membership/SSO/policy/module/
   integration configuration → launch gate → Control Plane.

9. Institutional buyer:
   Lead capture → qualification → discovery/pilot scope → trust/procurement
   workspace → implementation handoff.

10. Implementation lead:
    Tenant/implementation workspace → migration and configuration tasks
    → readiness gate → go-live handoff.

11. Partner/developer:
    Account → agreement/terms → sandbox → scoped app credentials
    → valid API event → review/certification path.

12. Semester employee:
    Employee provisioning → required training → scoped workspace
    → first role action → access review cadence.

==
ROLE WORKSPACES AND ROUTES
==

Use one canonical app shell, navigation model, design system, identity system,
policy system, audit system, and workflow engine.

Do not make roles share generic dashboards if they have fundamentally different
daily jobs. Do not create separate products.

Reconcile/build role-aware workspaces such as:

Public:
/
 /product
 /solutions
 /pricing

```
 /pilot
 /demo
 /trust
 /developers
 /partners
 /careers
 /auth
 /onboarding
 /download
 /app

Student:
 /app/t/:tenantSlug/student/today
 /app/t/:tenantSlug/student/actions
 /app/t/:tenantSlug/student/plan
 /app/t/:tenantSlug/student/registration
 /app/t/:tenantSlug/student/courses
 /app/t/:tenantSlug/student/study
 /app/t/:tenantSlug/student/support
 /app/t/:tenantSlug/student/campus
 /app/t/:tenantSlug/student/community
 /app/t/:tenantSlug/student/career
 /app/t/:tenantSlug/student/account
 /app/t/:tenantSlug/student/privacy
 /app/t/:tenantSlug/student/settings

Faculty:
 /app/t/:tenantSlug/faculty/courses
 /app/t/:tenantSlug/faculty/courses/:courseId
 /app/t/:tenantSlug/faculty/teaching
 /app/t/:tenantSlug/faculty/assessments
 /app/t/:tenantSlug/faculty/gradebook
 /app/t/:tenantSlug/faculty/office-hours
 /app/t/:tenantSlug/faculty/course-policy

Advisor:
 /app/t/:tenantSlug/advisor/caseload
 /app/t/:tenantSlug/advisor/students/:studentId
 /app/t/:tenantSlug/advisor/appointments
 /app/t/:tenantSlug/advisor/referrals
 /app/t/:tenantSlug/advisor/outreach
 /app/t/:tenantSlug/advisor/outcomes

Registrar and Academic Operations:
 /app/t/:tenantSlug/registrar/overview
 /app/t/:tenantSlug/registrar/terms
 /app/t/:tenantSlug/registrar/catalog
 /app/t/:tenantSlug/registrar/sections
 /app/t/:tenantSlug/registrar/registration
 /app/t/:tenantSlug/registrar/holds
 /app/t/:tenantSlug/registrar/overrides
 /app/t/:tenantSlug/registrar/records
 /app/t/:tenantSlug/registrar/degree-audit
 /app/t/:tenantSlug/registrar/graduation
 /app/t/:tenantSlug/registrar/reconciliation
```

```
Institution:
 /app/t/:tenantSlug/institution/overview
 /app/t/:tenantSlug/institution/academic-operations
 /app/t/:tenantSlug/institution/student-success
 /app/t/:tenantSlug/institution/student-accounts
 /app/t/:tenantSlug/institution/campus-services
 /app/t/:tenantSlug/institution/community-safety
 /app/t/:tenantSlug/institution/career
 /app/t/:tenantSlug/institution/implementation
 /app/t/:tenantSlug/institution/identity
 /app/t/:tenantSlug/institution/integrations
 /app/t/:tenantSlug/institution/governance
 /app/t/:tenantSlug/institution/assurance
 /app/t/:tenantSlug/institution/audit
 /app/t/:tenantSlug/institution/portability

Semester Operations:
 /app/ops
 /app/ops/inbox
 /app/ops/work
 /app/ops/approvals
 /app/ops/tenants
 /app/ops/customers
 /app/ops/pilots
 /app/ops/implementations
 /app/ops/customer-health
 /app/ops/support
 /app/ops/revenue
 /app/ops/billing
 /app/ops/finance
 /app/ops/growth
 /app/ops/onboarding
 /app/ops/product
 /app/ops/releases
 /app/ops/rollouts
 /app/ops/integrations
 /app/ops/migrations
 /app/ops/security
 /app/ops/privacy
 /app/ops/access-reviews
 /app/ops/break-glass
 /app/ops/incidents
 /app/ops/reliability
 /app/ops/audit
 /app/ops/risks
 /app/ops/vendors
 /app/ops/people
 /app/ops/board
 /app/ops/partners
 /app/ops/developers

Routes are not authority. Enforce authorization at:
- UI route boundary
- server action/API/RPC
- database RLS policy
- file/storage policy
```

- workflow command
- export
- AI retrieval/tool/action boundary

==
CANONICAL CONTROLLED ACTION PATTERN
==

Every high-impact operation uses:

Request
→ authenticate identity
→ resolve tenant, role, scope, and relationship
→ evaluate capability, policy, consent, classification, entitlement, and
  workflow state
→ fetch authoritative data
→ show impact preview
→ require step-up authentication and/or approval when needed
→ run idempotent server-side command
→ write immutable audit event
→ write domain/outbox event
→ update projections/read models
→ notify affected user(s)
→ issue clear receipt/history
→ expose rollback/remediation path where permitted

Use this pattern for:
- registration
- holds/overrides
- grade release/change
- academic-record changes
- degree audit/graduation
- credits/refunds/write-offs
- subscription/entitlement changes
- policy changes
- role/capability grants
- break-glass access
- SSO/SCIM/integration changes
- AI policy/model/provider changes
- data export/deletion
- migration/cutover
- feature rollout/kill switch
- security or incident actions

==
CROSS-ROLE WORKFLOWS
==

Implement or reconcile these canonical workflows.

1. Student ↔ Faculty
Course content, assignments, assessment, feedback, office hours, grade release.

2. Student ↔ Advisor
Plan, check-in, appointment, referral, outreach, consent-aware collaboration.

3. Student ⇔ Registrar
Registration readiness, holds, enrollment, waitlist, overrides, records, degree audit, graduation.

4. Student ⇔ Student Accounts
Charges, credits, statements, payment plans, support, approved handoffs.

5. Student ⇔ Campus Services
Housing, dining, library, spaces, events, accessibility, wellness/safety handoffs, local services.

6. Student ⇔ Career/Alumni/Employer
Skills evidence, opportunities, portfolio, mentoring, applications, outcomes.

7. Student ⇔ Family
Explicit, scoped, time-limited, revocable consent grants only.

8. Student ⇔ Community
Membership, study groups, events, reporting, moderation, appeals, safety.

9. Institution ⇔ Semester
Pilot, contract, tenant setup, implementation, migration, training, launch, support, QBR, renewal, expansion, portability/offboarding.

10. Semester ⇔ Partner/Developer
Sandbox, scopes, credentials, certification, marketplace installation, monitoring, versioning, incident and deprecation handling.

==========
AI PLATFORM FOR ALL ROLES
==========

For every AI interaction:

user request
→ authenticated identity
→ tenant
→ role/capability
→ resource relationship
→ context/course/workspace
→ data classification
→ consent
→ policy
→ permitted provider/model
→ approved source retrieval
→ allowed tools
→ response/action
→ source/provenance/citation
→ uncertainty and limitation labels
→ human/official workflow where required
→ audit, evaluation, cost, and incident telemetry

Create/reconcile role-specific AI policy configurations.

Examples:
- Student: study, planning, approved institutional guidance.

- Faculty: course preparation and permitted pedagogical assistance.
- Advisor: permitted caseload summarization and workflow support.
- Registrar: explanation/drafting, never autonomous record action.
- Institution admin: configuration assistance, never autonomous policy change.
- Semester support: minimal-context support assistance.
- Developer: documentation and sandbox help.
- Finance: analysis/drafting, never autonomous financial approvals.

==
OFFBOARDING, PORTABILITY, AND LIFECYCLE
==

Implement role-change and offboarding workflows for:

- Student graduation, leave, transfer, withdrawal, and alumni transition.
- Faculty/TA end-of-term access sunset.
- Advisor caseload reassignment.
- Registrar/admin role transfer.
- Guardian consent expiration/revocation.
- Institutional customer cancellation/portability.
- Partner/developer app deprecation.
- Semester employee offboarding.
- Board-member access expiry.

Every offboarding workflow requires:

- Trigger and owner
- Data/record authority assessment
- Retention/legal-hold check
- Open-work ownership transfer
- Session and credential revocation
- Membership/capability removal
- Service-account/API/webhook handling where applicable
- Export/portability actions
- Archive/deletion schedule
- Final audit event
- Completion evidence

==
P0/P1/P2 IMPLEMENTATION PRIORITY
==

P0 — Build first to operate a credible registration-readiness pilot:

1. Canonical account, profile, tenant, membership, role, and capability model.
2. Server-side policy/consent/classification checks.
3. Audit-event standard and audit explorer.
4. Tenant-aware AppShell, role-aware navigation, forbidden/empty/loading/error
   states, responsive behavior, and shared design tokens.
5. Student self-serve and institution-linked account flows.
6. Institution SSO/invitation/account-linking flow.
7. Student onboarding: goal → term/registration context → Today → first action.
8. Advisor basic caseload, appointment, referral, and support workflow.
9. Registrar registration-readiness workflow: terms, holds, eligibility,
   prerequisite/capacity status, controlled enrollment handoff or transition.
10. Institution admin: tenant configuration, feature/module state,

memberships, roles, and basic policy configuration.
11. Semester Operations: Inbox, My Work, Approvals, Tenant 360, Customer 360, pilot/implementation tracking, support queue.
12. Integration registry, source/freshness labels, sync health, reconciliation.
13. Controlled rollout/feature flags/kill switches and incident runbook.
14. Activation/first-value analytics that exclude sensitive record content.
15. Minimum docs, tests, release gates, and support procedures.

P1 — Build after PO works safely:

- Faculty/TA Course Studio and assessment workflows.
- Student-success leadership and cohort views.
- Student account/payment-plan operational workflows.
- Campus-services workflows.
- Career/employer/alumni workflows.
- Family/guardian consent workspace.
- Community moderation/safety/appeals.
- Full privacy/security/accessibility governance consoles.
- Advanced institutional implementation/migration workflows.
- Native growth, sales, customer success, lifecycle, and finance consoles.
- AI governance/evaluation/incident workflows.
- PWA/optional native mobile continuity.

P2 — Build only after earlier domains have real evidence:

- Authoritative registrar/academic-record cutover factory.
- Full native financial operations/embedded-finance progression.
- Marketplace/partner ecosystem at scale.
- Global localization, regional configurations, and residency controls.
- Multi-institution network and lifelong credential portability.
- Advanced AI workflow agents with controlled, evaluated, reversible actions.
- Direct regulated-payment expansion only after dedicated legal, finance, treasury, security, fraud, bank/acquirer, and compliance readiness.

==========
DOCUMENTATION TO CREATE
==========

Create or reconcile:

docs/roles/ROLE_SYSTEM_MASTER_MAP.md
docs/roles/ROLE_CATALOG.md
docs/roles/ROLE_LIFECYCLE_CATALOG.md
docs/roles/ROLE_WORKSPACE_CATALOG.md
docs/roles/ROLE_CAPABILITY_MATRIX.md
docs/roles/ROLE_TENANT_SCOPE_MATRIX.md
docs/roles/ROLE_CROSS_WORKFLOW_MAP.md
docs/roles/ROLE_ONBOARDING_CATALOG.md
docs/roles/ROLE_OFFBOARDING_CATALOG.md
docs/roles/ROLE_ROUTE_CATALOG.md
docs/roles/ROLE_DATA_ACCESS_MATRIX.md
docs/roles/ROLE_AI_POLICY_MATRIX.md
docs/roles/ROLE_SUPPORT_ESCALATION_MAP.md
docs/roles/ROLE_READINESS_SCORECARD.md
docs/roles/ROLE_SYSTEM_GAP_REGISTER.md
docs/roles/ROLE_SYSTEM_EXECUTION_BACKLOG.md

```
docs/platform/CANONICAL_IDENTITY_AND_ACCESS_MODEL.md
docs/platform/CAPABILITY_AND_SCOPE_EVALUATION.md
docs/platform/CONTROLLED_ACTION_PATTERN.md
docs/platform/WORKFLOW_AND_APPROVAL_ENGINE.md
docs/platform/AUDIT_EVENT_STANDARD.md
docs/platform/OUTBOX_AND_PROJECTION_MODEL.md
docs/platform/TENANT_ISOLATION_AND_RLS.md
docs/platform/DATA_AUTHORITY_AND_SOURCE_STATUS.md
docs/platform/CONSENT_CLASSIFICATION_AND_RETENTION.md
docs/platform/AI_ROLE_GOVERNANCE.md

docs/finish-line/ROLE_SYSTEM_RELEASE_GATES.md
docs/finish-line/ROLE_SYSTEM_TEST_MATRIX.md
docs/finish-line/ROLE_SYSTEM_EVIDENCE_REGISTER.md
docs/finish-line/ROLE_SYSTEM_RISK_REGISTER.md
docs/finish-line/ROLE_SYSTEM_90_DAY_PLAN.md

================================================================
TEST AND RELEASE REQUIREMENTS
================================================================

For every role and workspace, add and run:

- Unit tests
- Integration tests
- Route authorization tests
- API/RPC authorization tests
- RLS/tenant-isolation tests
- Capability/scope tests
- Consent and policy tests
- Workflow state/approval tests
- Idempotency tests for commands
- Audit-event tests
- Accessibility tests
- Keyboard-only tests
- Mobile/responsive tests
- Error/empty/loading/forbidden state tests
- Session/recovery/offboarding tests
- AI policy/tool-permission tests where AI is present
- Analytics privacy tests
- Migration/rollback tests where data authority changes
- Build/typecheck/lint tests
- Manual role-based acceptance scripts for high-risk workflows

Use representative test fixtures that contain no production personal data.

================================================================
BRANCH PLAN
================================================================

Create and work in small, reviewable branches in this order:

audit/role-system-reconciliation
feat/canonical-identity-membership-capabilities
feat/policy-consent-classification-audit-foundation
```

```
design/role-aware-appshell-navigation
feat/student-onboarding-and-first-value
feat/institution-sso-invites-account-linking
feat/advisor-student-success-p0
feat/registrar-registration-readiness-p0
feat/institution-control-plane-p0
feat/semester-ops-command-center-p0
feat/integration-source-status-reconciliation-p0
feat/release-rollout-incident-controls-p0
feat/faculty-course-studio-p1
feat/student-finance-campus-career-family-community-p1
feat/trust-privacy-security-accessibility-consoles-p1
feat/company-os-growth-revenue-success-finance-people-p1
feat/developer-partner-marketplace-p2
feat/role-system-finish-line-evidence

Do not merge a feature branch unless:
- canonical data models are used
- RLS/policy checks exist
- tests pass
- accessibility review is complete
- monitoring/audit are present where required
- documentation is updated
- rollback is defined for high-impact change
- readiness state is honestly marked

================================================================
COMMANDS AND QUALITY GATES
================================================================

First inspect package scripts. Then run applicable commands without inventing
scripts that do not exist:

npm ci
npm run lint
npm run test
npm run build
npm run typecheck
npm run test:a11y
npm run test:integration
npm run test:rls
npm run test:e2e
npm run security
npm run secrets

Use Supabase safely:

supabase login
supabase link --project-ref lzrqvlugnawcgywkhqlz
supabase migration list

Never blindly apply broad production migrations.
Use development/staging/branch environments, migration review, RLS tests,
reconciliation, backup validation, and rollback preparation before production
changes.
```

```
================================================================
REQUIRED RESPONSE FORMAT AFTER EACH IMPLEMENTATION PHASE
================================================================

Return exactly:

1. Phase and branch
2. Objective
3. Current-state findings
4. Canonical systems reused
5. Duplicate/stale systems found
6. Files created or changed
7. Routes/screens added or changed
8. Database migrations/RPCs/functions/policies changed
9. Role/capability/policy controls implemented
10. Audit and evidence behavior
11. Onboarding and lifecycle behavior
12. AI governance behavior, if relevant
13. Security/privacy/tenant-isolation work
14. Accessibility/responsive work
15. Integrations or migration bridges affected
16. Tests run and exact results
17. Build/lint/typecheck results
18. Current readiness classification
19. Risks, assumptions, and required human review
20. Rollback/remediation plan
21. Next dependency-ordered actions

Do not say “complete” without showing implementation and evidence.

================================================================
IMMEDIATE FIRST TASK
================================================================

Start with only:

audit/role-system-reconciliation

Do not broadly code product features yet.

Read the attached PDFs and repository. Then create the role-system audit and
return:

1. Actual existing role/identity/tenant/capability systems
2. Actual existing routes and role workspaces
3. Actual existing Supabase tables, RLS policies, and functions involved
4. Duplicate or conflicting role/permission models
5. Current student, faculty, advisor, registrar, institution-admin, and
   Semester-operator flows
6. Missing PO pilot workflows
7. Security/RLS/high-risk gaps
8. Accessibility and mobile gaps
9. Exact PO implementation sequence
10. First 25 dependency-ordered tasks
11. First 10 branches with acceptance criteria
12. Updated documentation list and locations
```

Recommended first local commands

Run these in the repository root before giving the output back to the agent:

```
git clone https://github.com/harrisonjrubin7-cmyk/semester.git
cd semester

git checkout -b audit/role-system-reconciliation

npm ci
npm run

# If the main application is in app/
cd app
npm ci
npm run

npm run lint
npm run test
npm run build

# Run only if scripts exist
npm run typecheck
npm run test:integration
npm run test:rls
npm run test:a11y
npm run test:e2e
npm run security
npm run secrets
```

First acceptance criteria

Do not allow the agent to claim that the role system is implemented until it has produced, at minimum:

```
[ ] One canonical account/profile model
[ ] One canonical tenant/membership model
[ ] One role/capability/scope evaluation model
[ ] Server-side enforcement plus RLS enforcement
[ ] A capability matrix for every PO role
[ ] Role-aware routing and navigation
[ ] Student, advisor, registrar, institution-admin, and Semester-operator
    first-value journeys
[ ] PO cross-role workflow map
[ ] Audit and approval patterns for sensitive actions
[ ] Onboarding progress that survives browser/PWA/native transitions
[ ] Support/escalation and offboarding flows
[ ] Accessibility, mobile, security, and testing evidence
[ ] Honest readiness labels for every domain
```

The most important instruction to retain is: **do not build many dashboards. Build one native Semester system with different role workspaces over one governed, authoritative platform.**

⁂

1. list-every-artifact-and-thing-for-claud-to-genrate.pdf
2. expand-evan-further-provide-prompt-and-comand-to-e.pdf
3. Command-center-screen_Company-operations-command-c.pdf
4. expand-and-devlop-how-i-build-these-into-semester-2.pdf
5. now-break-down-entire-semester-infstucture-and-sys.pdf
6. depect-semester-flow-for-every-role-wether-you-wor.pdf